



Основы радиоэлектроники и связи

Лекция 2. Инструменты программирования и визуализации

Антон Штанюк

КТПП::ИРИТ::НГТУ / 2026-08-26

Использование языка программирования Python

Библиотека визуализации Matplotlib

Web-приложение для работы с документами/программами Jupyter Lab



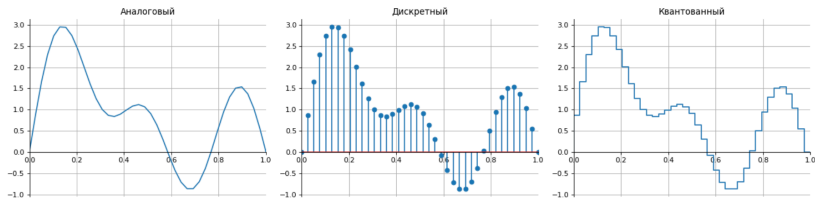
Использование языка программирования Python

Почему Python

- Синтаксически простой ("современный бейсик")
- Современный
- Мегапопулярный
- Готовые библиотеки для множества задач (включая ИИ)
- Кроссплатформенный

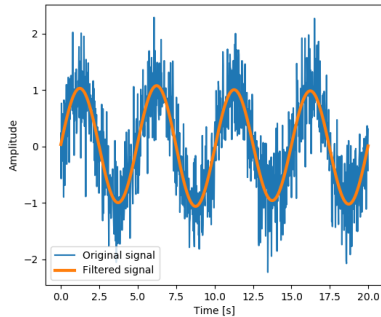
Особенность Python

В Python есть мощный графический пакет **matplotlib**, с помощью которого легко визуализировать сигналы



Особенность Python

Также в Python есть мощные пакеты **scipy.fft**, **scipy.signal**, с помощью которого легко сигналы обрабатывать, вычислять спектры и строить фильтры.



Особенность Python

Язык интерпретируемый, но с предварительной компиляцией. То есть, не нужно сначала строить исполняемый файл, а потом запускать его. Самый типичный способ запуска: из командной строки.

Запуск программы prog.py

```
python prog.py
```

Особенность Python

Другой способ запуска: Jupyter Notebook

Jupyter Optical Coherence Tomography-Copy1 Last Checkpoint: Last Sunday at 6:14 PM (autosaved)

File Edit View Insert Cell Kernel Navigate Widgets LaTeX Envs Help

Contents

- 1 Optical Coherence Tomography
 - 1.1 Imports, preliminaries, def
 - 1.2 Imaging system - overview
 - 1.3 OCT Theory - overview
 - 1.3.1 Comments and calc
 - 1.3.1.1 Resolution "back"
 - 1.3.1.2 Scan depth "back"
 - 1.3.1.3 Scaling of coheren
 - 1.3.2 Time Domain OCT (TC
 - 1.3.2.1 Detection-banded
 - 1.3.2.2 TDOCT: SNR and
 - 1.3.3 Fourier Domain OCT
 - 1.3.3.1 Impact of finite age
 - 1.3.3.2 Interlude: Finite aa
 - 1.3.3.3 Impact of finite nur
 - 1.3.3.4 FDOCT: SNR and
 - 1.3.4 Spectral domain sweep
 - 1.3.4.1 SSDOCT: SNR and I
 - 1.4 Simulation
 - 1.5 Potential laser sources

1.3.3 Fourier Domain OCT (FDOCT)

In FDOCT, the different wavelengths are collected on a spectrometer, with N_{det} pixels, and spectral resolution $\delta\lambda$.

Returning again to Eq. (8) (see, e.g., Izatt and Choma (Izatt J.A., Choma M.A. (2008) Theory of Optical Coherence Tomography. In: Drexler W., Fujimoto J.G. (eds) Optical Coherence Tomography, Biological and Medical Physics, Biomedical Engineering. Springer, Berlin, Heidelberg. doi: https://doi.org/10.1007/978-3-540-77550-0_2; alternate link: https://www.researchgate.net/publication/226178102_Theory_of_Optical_Coherence_Tomography/download).

$$I_D(k) = \frac{Q}{4} S(k) \left[R_E + \sum_{k=1}^N R_k \right] \quad \text{"DC terms"}$$

$$+ \frac{Q}{2} S(k) \left[\sum_{k=1}^N \sqrt{R_E R_k} \cos[2k(x_E - x_k)] \right] \quad \text{"Cross - correlation terms"}$$

$$+ \frac{Q}{2} S(k) \left[\sum_{k=1}^N \sqrt{R_k R_m} \cos[2k(x_k - x_m)] \right] \quad \text{"Autocorrelation terms"}$$

In the FDOCT configuration, x_E is held fixed.

```
In [23]: lambda_0 = 1.55e6
k_0 = 2.0*np.pi/lambda_0
Dlambda_0 = 0.1e6
dk = 2.0*np.pi*Dlambda_0/lambda_0**2.0

k_range = np.linspace(-3.0*dk*k_0, +3.0*dk*k_0, 10000)

TD_OCT_signal = 0.25*0.5*(np.exp(-(k_range - k_0)/DK)**2.0) \
+ 0.5*np.sqrt(0.5**2.0*-4)*(np.exp(-(k_range - k_0)/DK)**2.0) \
*np.cos(2.0*k_range*(50.0)) \
+ 0.5*np.sqrt(0.5**2.0*-4)*(np.exp(-(k_range - k_0)/DK)**2.0) \
*np.cos(2.0*k_range*(100.0))

In [26]: fig_disp
Out[26]:
```

Особенность Python

После установки Python на ПК важно убедиться, что путь к интерпретатору python добавлен в системную переменную **path**

Состав программы

Типичная программа включает три области:

1. Импорт библиотек (пакетов)
2. Описание своих функций
3. Главный код, использующий функции

Пример функции: манчестерское кодирование

```
def manchester_encode(binary_signal):
```

```
    """
```

Кодирует одну битовую строку по алгоритму Манчестера.

Предполагается, что входная строка содержит только '0' и '1'.

Каждый бит преобразуется:

'0' -> '01'

'1' -> '10'

Аргументы:

binary_signal (str): Входная строка из '0' и '1'.

Возвращает:

str: Строка с Манчестер-кодированными уровнями.

```
    """
```

```
    if not binary_signal:
```

```
        return ""
```

Пример функции: манчестерское кодирование

```
encoded_bits = []  
for bit in binary_signal:  
    if bit == '0':  
        encoded_bits.append('01')  
    else: # Предполагается, что если не '0', то это '1'  
        encoded_bits.append('10')  
return ''.join(encoded_bits)  
# здесь заканчивается код функции  
input_str = "0101010101"  
output_str = manchester_encode(input_str)  
print(output_str)
```

Синтаксические особенности

- Очень важны отступы в начале строк (зато нет лишних скобок)
- Основная структура данных - список (определяется как `[]`)
- Динамическая типизация (тип определяется из контекста использования)

Подключение библиотек (внешний импорт)

```
import numpy as np
import matplotlib
import matplotlib.pyplot as plt
import random
from collections import Counter
import math
```

Об установке пакетов

Для того, чтобы подключить библиотеку ее нужно предварительно скачать и проверить на совместимость

Для этого существует специальная команда

Установка пакета numpy

```
pip install numpy
```

Для установки любого пакета достаточно заменить **numpy** на другое имя

NumPy

NumPy - важнейшая библиотека для представления и обработки числовых данных.

Вот некоторые важнейшие направления для ее использования:

- Научные и инженерные вычисления;
- Машинное обучение и искусственный интеллект;
- Обработка данных;
- Визуализация данных;
- Высокопроизводительные вычисления;

Массивы в NumPy

Массив - главная структура данных в NumPy. В отличие от списков, все элементы массива должны иметь один тип

```
a = np.array([1, 2, 3, 4, 5, 6])  
b = np.array([[1, 2, 3, 4], [5, 6, 7, 8], [9, 10, 11, 12]])  
c = np.array([[1, 2, 3, 4],  
              [5, 6, 7, 8],  
              [9, 10, 11, 12]])
```




Библиотека визуализации Matplotlib

Установка

Как и все другие библиотеки, `matplotlib` устанавливается стандартным образом:

```
pip install matplotlib
```

И внутри файла мы обычно импортируем инструменты для рисования так:

```
import matplotlib.pyplot as plt
```

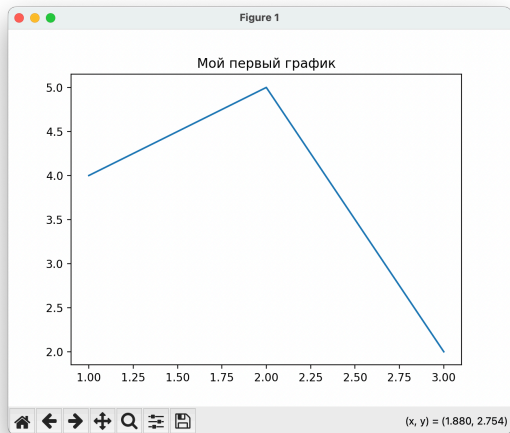
Два важных объекта библиотеки

- **Figure (Фигура)**: Это мольберт, чистый холст или окно программы. Сам по себе холст не содержит графиков, это просто пространство. На одном холсте (Figure) может располагаться как один рисунок, так и целая сетка из множества рисунков.
- **Axes (График)**: Это непосредственно сам рисунок на холсте. Именно здесь живут ваши данные, оси координат (X и Y), легенда и подписи.

Первый подход к рисованию (процедурный)

```
# Быстро, просто, но не гибко  
plt.plot([1, 2, 3], [4, 5, 2])  
plt.title("Мой первый график")  
plt.show()
```

Результат визуализации



Второй подход к рисованию (объектно-ориентированный)

```
# Быстро, просто, но не гибко  
plt.plot([1, 2, 3], [4, 5, 2])  
plt.title("Мой первый график")  
plt.show()
```

Визуализация непрерывного сигнала (1)

```
t = np.linspace(0, 1.0, 500)  # 500 точек от 0 to 1 секунды

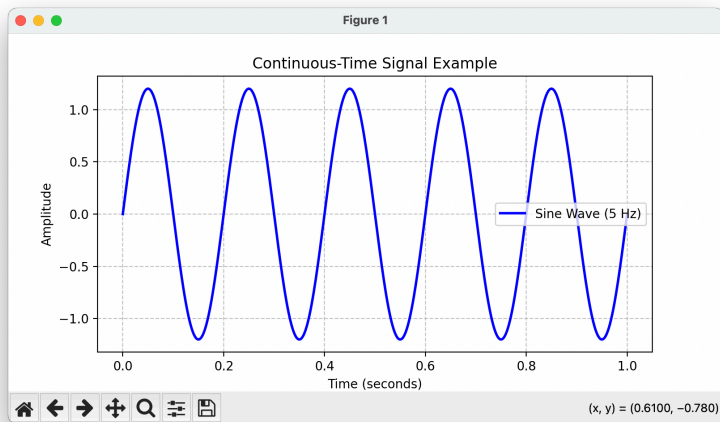
frequency = 5  # 5 Hz
amplitude = 1.2
signal = amplitude * np.sin(2 * np.pi * frequency * t)

plt.figure(figsize=(8, 4))
plt.plot(t, signal, label='Sine Wave (5 Hz)',
         color='blue', linewidth=2)
```

Визуализация непрерывного сигнала (2)

```
plt.title('Пример непрерывного сигнала')  
plt.xlabel('t, с')  
plt.ylabel('Амплитуда')  
plt.grid(True, linestyle='--', alpha=0.7)  
plt.legend()  
plt.show()
```

Результат визуализации



Визуализация дискретного сигнала (1)

```
n = np.arange(0, 30)  # 30 отсчетов

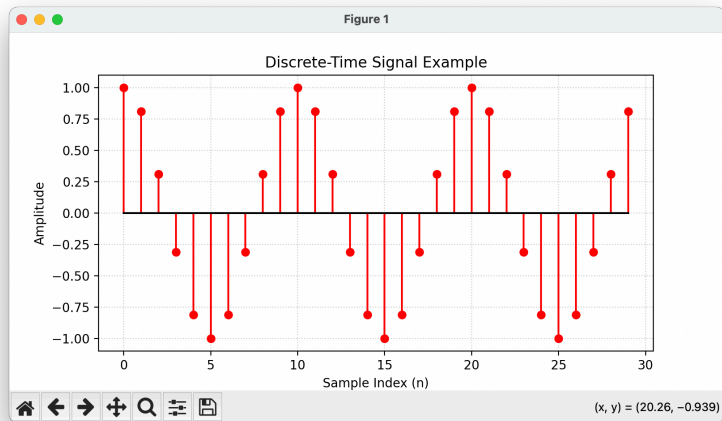
discrete_signal = np.cos(2 * np.pi * 0.1 * n)

plt.figure(figsize=(8, 4))
plt.stem(n, discrete_signal, linefmt='red',
         markerfmt='ro', basefmt='black')
```

Визуализация дискретного сигнала (2)

```
plt.title('Пример дискретного сигнала')  
plt.xlabel('Индекс (n)')  
plt.ylabel('Амплитуда')  
plt.grid(True, linestyle=':', alpha=0.6)  
plt.show()
```

Результат визуализации



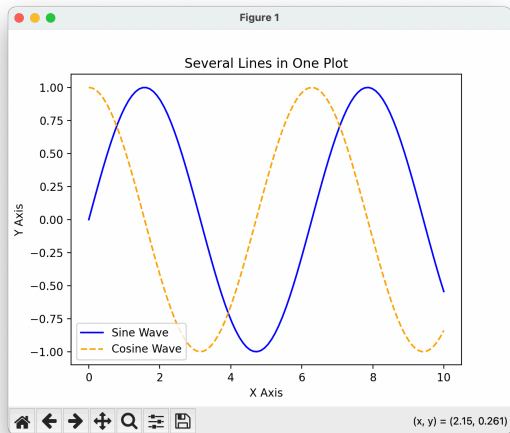
Несколько зависимостей на одном графике

```
x = np.linspace(0, 10, 100)

# Plotting multiple lines on the same axes
plt.plot(x, np.sin(x), label='Sine Wave', color='blue')
plt.plot(x, np.cos(x), label='Cosine Wave', color='orange',
         linestyle='--')

plt.title('Several Lines in One Plot')
plt.xlabel('X Axis')
plt.ylabel('Y Axis')
plt.legend() # Displays the labels
plt.show()
```

Результат визуализации



Несколько графиков на одном рисунке

```
import matplotlib.pyplot as plt
import numpy as np
import math

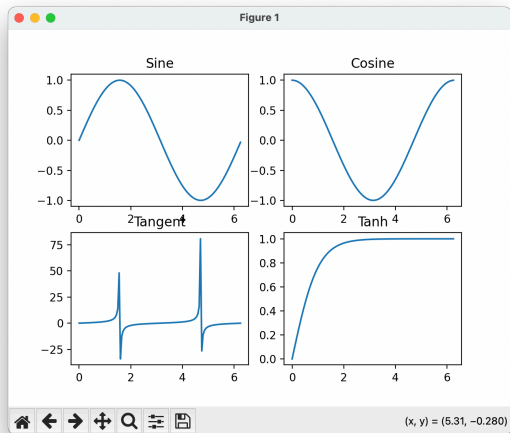
x = np.arange(0, 2*math.pi, 0.05)
y1, y2, y3, y4 = np.sin(x), np.cos(x), np.tan(x), np.tanh(x)

fig, ax = plt.subplots(2, 2)

ax[0,0].plot(x, y1); ax[0,0].set_title("Sine")
ax[0,1].plot(x, y2); ax[0,1].set_title("Cosine")
ax[1,0].plot(x, y3); ax[1,0].set_title("Tangent")
ax[1,1].plot(x, y4); ax[1,1].set_title("Tanh")

plt.show()
```

Результат визуализации



Ступенчатый график

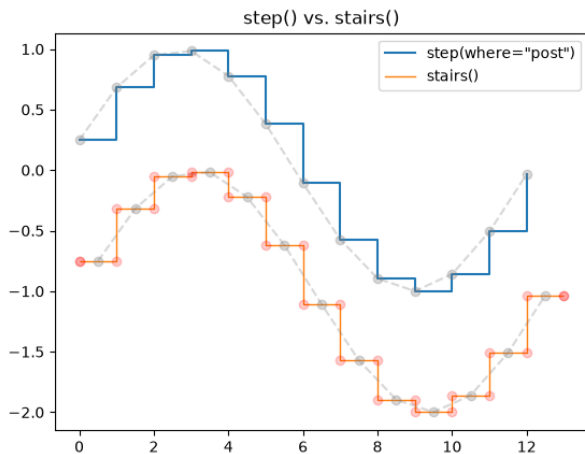
```
bins = np.arange(14)
centers = bins[:-1] + np.diff(bins) / 2
y = np.sin(centers / 2)

plt.step(bins[:-1], y, where='post', label='step(where="post")')
plt.plot(bins[:-1], y, 'o--', color='grey', alpha=0.3)

plt.stairs(y - 1, bins, baseline=None, label='stairs()')
plt.plot(centers, y - 1, 'o--', color='grey', alpha=0.3)
plt.plot(np.repeat(bins, 2), np.hstack([y[0], np.repeat(y, 2), y[-1]]),
         'o', color='red', alpha=0.2)

plt.legend()
plt.title('step() vs. stairs()')
plt.show()
```

Результат визуализации



Интервальный график

```
x = np.arange(14)
```

```
y = np.sin(x / 2)
```

```
plt.step(x, y + 2, label='pre (default)')
```

```
plt.plot(x, y + 2, 'o--', color='grey', alpha=0.3)
```

```
plt.step(x, y + 1, where='mid', label='mid')
```

```
plt.plot(x, y + 1, 'o--', color='grey', alpha=0.3)
```

```
plt.step(x, y, where='post', label='post')
```

```
plt.plot(x, y, 'o--', color='grey', alpha=0.3)
```

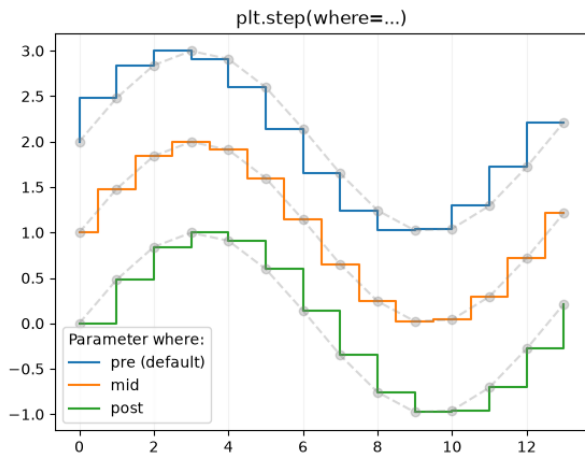
```
plt.grid(axis='x', color='0.95')
```

```
plt.legend(title='Parameter where:')
```

```
plt.title('plt.step(where=...)')
```

```
plt.show()
```

Результат визуализации



3D график

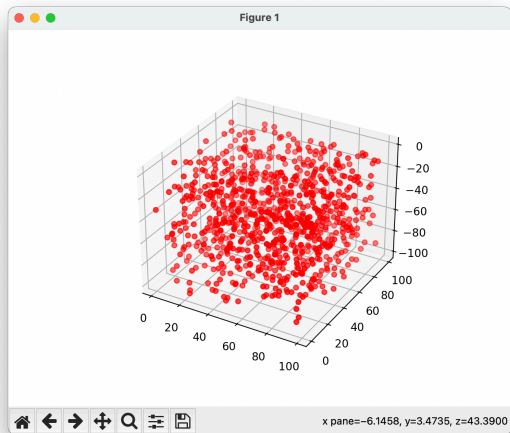
```
np.random.seed(29)
d = np.random.randint(1, 100, size=(3, 30, 30))
print(d)
z, x, y = d

fig = plt.figure()
ax = plt.axes(projection='3d')
ax.scatter(x, y, -z, zdir='z', c= 'red')
plt.show()
```

Вывод данных

```
[[[86 35 41 ... 4 47 77]  
  [79 27 14 ... 61 83 48]  
  [36  5  5 ... 15 10 85]  
  ...  
  [36 28 29 ... 73  1 51]  
  [95 75 68 ... 28 73 67]  
  [63  5 32 ... 59 97 49]]
```

Результат визуализации



Сохранение графика в файл

```
...  
fig.savefig('my_awesome_plot.png', dpi=300, bbox_inches='tight')  
...
```

Важное правило! Сначала сохраняем (fig.savefig), и только в самом конце показываем (plt.show())



Web-приложение для работы с документами/программами Jupyter Lab

Установка

Как и все другие библиотеки, jupyter lab устанавливается стандартным образом:

```
pip install jupyterlab
```

Запуск

Для запуска необходимо в командной строке перейти в каталог с документами и запустить приложение:

```
jupyter lab .
```

Точка в данном примере означает запуск для текущего каталога. После старта на экране высветится URL для открытия страницы в браузере

URL для открытия страницы

117.

```
[C 2026-09-09 17:43:33.495 ServerApp]
```

To access the server, open this file

`file:///Users/anton/Library/Jupy`

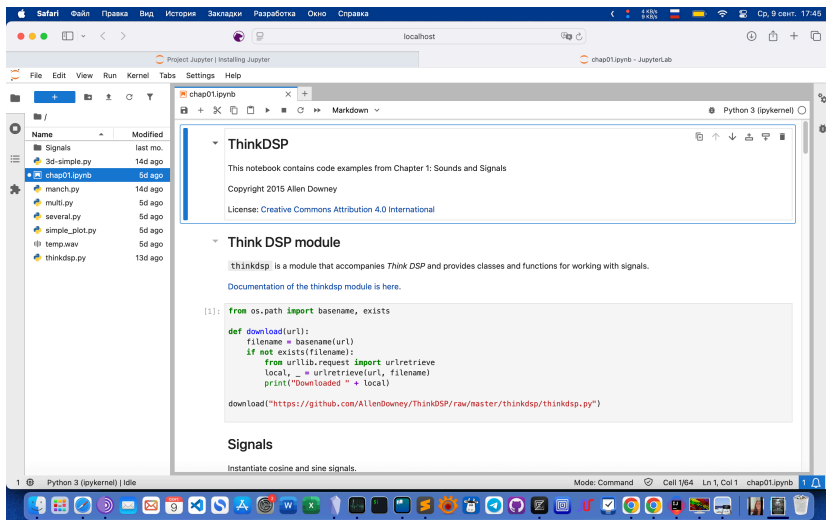
Or copy and paste one of these URLs:

`http://localhost:8889/lab?token=`

`http://127.0.0.1:8889/lab?token=`

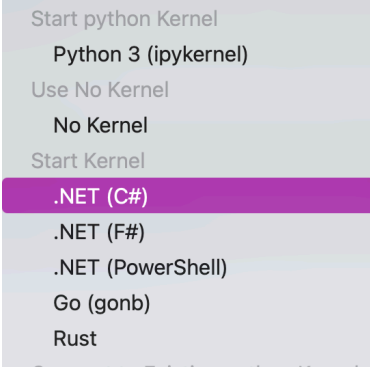
```
[I 2026-09-09 17:43:33.903 ServerApp] Sk
```

Главное окно



Замечания по настройке

Jupyter Lab требует указания вычислительных ядер, которые обрабатывают ячейки документа с кодом. По-умолчанию подразумевается ядро **Python**, но

A screenshot of the Jupyter Lab kernel selection menu. The menu is a vertical list of options. The first four options are grouped under the heading 'Start python Kernel'. The next two options are grouped under 'Use No Kernel'. The remaining options are grouped under 'Start Kernel'. The option '.NET (C#)' is highlighted with a purple background.

- Start python Kernel
 - Python 3 (ipykernel)
- Use No Kernel
 - No Kernel
- Start Kernel
 - .NET (C#)
 - .NET (F#)
 - .NET (PowerShell)
 - Go (gonb)
 - Rust

можно подключать и другие языки.

Структура документа

Документ строится из ячеек, которые бывают двух типов:

1. Текст с разметкой (Markdown)
2. Выполняемый код

Ячейки можно добавлять/удалять/редактировать или выполнять (с кодом)

Список источников I